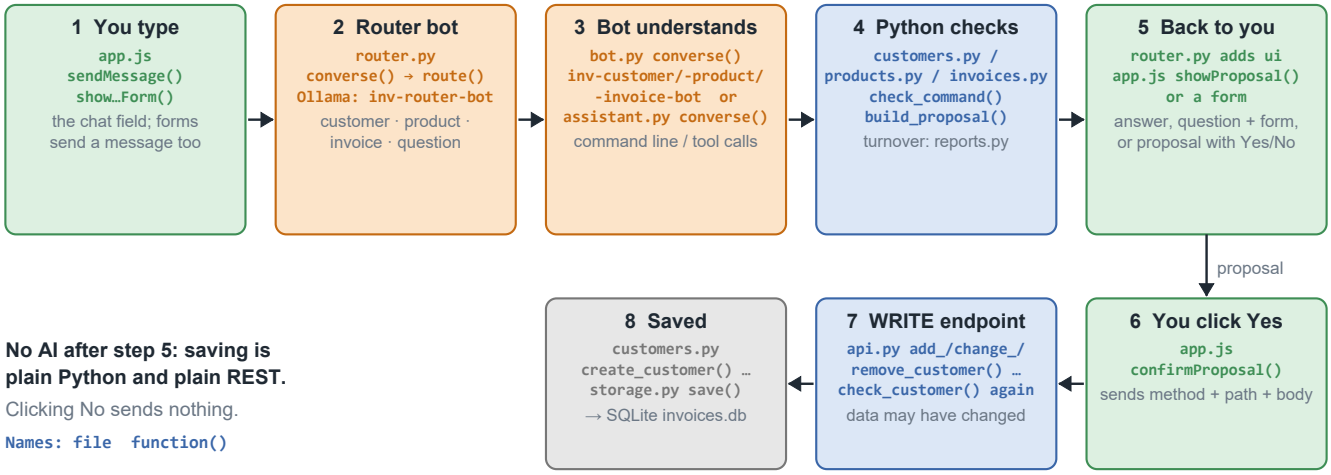
AI Invoice Assistant – how it works, on one page

A small invoicing program that runs on one computer: a **web page**, a **Python server** (FastAPI, app/api.py) and **local language models** (Ollama, llama3.2:3b). It keeps customers, products and invoices in a SQLite file and reports turnover. You use it through one chat field; small forms help you fill in what is missing.

The path of every message



No AI after step 5: saving is plain Python and plain REST.

Clicking No sends nothing.

`Names: file  function()`

Three rules

The models only understand language.	They turn a sentence into a route, a command line (CREATE_CUSTOMER {…}) or a tool call. Python finds records, checks values, adds totals and calculates turnover.
The chat never saves.	It ends in a *reply* or a *proposal*: a REST call that has been described but not sent. Only Yes sends it, and the WRITE endpoint checks it again.
Forms speak in chat messages.	A form sends a fixed sentence such as “customer: Karl Becker” to /chat, just like typed text. The waiting bot or Python reads it.

Who does what

Part	Job
static/app.js	The chat, the forms, Yes / No and the tables. Sends a proposal only after Yes.
app/api.py	The door: one function per URL. READ, WRITE (the only place that saves), CHAT, /help.
chat/router.py	Asks the router bot where a message belongs. It remembers which bot is waiting for an answer and opens the right form.
chat/bot.py	The customer, product and invoice bots. Reads their command line, catches invented names, and asks the entity module to check the command and build the proposal.
chat/assistant.py	Questions: a model with tools. Python answers the tools and writes invoice lists. It answers the turnover forms without any model.
chat/guide.py	The Help button: finds the 3 closest sections of the user guide and answers only from them.
entities/*.py	Customers, products, invoices: checks, find by name, create / update / delete.
core/reports.py · storage.py	Turnover = paid invoices, counted by due date · SQLite load / save, and the next number (CUS-013).

Safety nets

- **Two checks:** the same check function runs when the proposal is built and again when it is saved, so stale or edited data is refused (“Not saved: …”).
- **Python guards against model mistakes:** a name or number you never wrote → “Which customer do you mean?”, a negative price → the form, a paid invoice or a customer with invoices → refused.
- **Python writes the numbers:** lists, totals and turnover are written by Python, not copied by a model.

What is remembered where

Chats live only in the server's memory: the router's sessions and each bot's message history. A restart forgets them. The browser keeps one conversation id until the page is reloaded. Records live permanently in **data/invoices.db**. The details, with 19 step-by-step scenarios, are in **docs/dataflow_scenarios.pdf**.